

Digital Logic Design

Lecture Topic: Standard Forms

DR. SYED SHAHID SULTAN



In the previous lecture

- Boolean Functions

In the current lecture

- Standard Forms
- CLO 2: Simplify Boolean functions using postulates and K-maps

Canonical and Standard Forms

- Minterms and Maxterms
 - A *minterm*: an AND term consists of all literals in their normal form or in their complement form
 - For example, two binary variables x and y ,
 - $xy, xy', x'y, x'y'$
 - It is also called a *standard product*
 - n variables can be combined to form 2^n minterms
 - A *maxterm*: an OR term
 - It is also call a standard sum
 - For example,
 - $(x+y)$
 - 2^n maxterms
 - each maxterm is the complement of its corresponding minterm, and vice versa

Canonical and Standard Forms (cont.)

- m_j , the subscript j denotes the decimal equivalent of the binary number of the minterm designated.
- Each maxterm is obtained from an OR term of the n variables with each variable being unprimed.
- Each maxterm is the complement of its corresponding minterm and vice versa.

Table 2.3

Minterms and Maxterms for Three Binary Variables

x	y	z	Minterms		Maxterms	
			Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

Canonical and Standard Forms (cont.)

- An Boolean function can be expressed by
 - a truth table
 - sum of minterms
 - $f_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7$
 - $f_2 = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7$

Table 2.4
Functions of Three Variables

x	y	z	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Canonical and Standard Forms (cont.)

- The complement of a Boolean function

- the minterms that produce a 0

- $f_1' = m_0 + m_2 + m_3 + m_5 + m_6$
 $= x'y'z' + x'yz' + x'yz + xy'z + xyz'$

- $f_1 = (f_1')'$
 $= (x+y+z)(x+y'+z)(x+y'+z')(x'+y+z')(x'+y'+z)$
 $= M_0 M_2 M_3 M_5 M_6$

- Any Boolean function can be expressed as

- a sum of minterms
- a product of maxterms
- Boolean functions expressed as a sum of minterms or product of maxterms are said to be in *canonical form*.

Sum of minterms

- **Example**

- $F = A + B'C$
 $= A(B + B') + B'C$
 $= AB + AB' + B'C$
 $= AB(C + C') + AB'(C + C') + (A + A')B'C$
 $= ABC + ABC' + AB'C + AB'C' + A'B'C$
- $F = A'B'C + AB'C' + AB'C + ABC' + ABC$
 $= m_1 + m_4 + m_5 + m_6 + m_7$
- $F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$
- or, built the truth table first

Table 2.5

Truth Table for $F = A + B'C$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Product of maxterms

- Product of maxterms

- $x + yz = (x + y)(x + z)$
 $= (x+y+zz')(x+z+yy')$
 $= (x+y+z)(x+y+z')(x+y'+z)$

- $F = xy + x'z$
 $= (xy + x')(xy + z)$
 $= (x+x')(y+x')(x+z)(y+z)$
 $= (x'+y)(x+z)(y+z)$

- $x'+y = x' + y + zz'$
 $= (x'+y+z)(x'+y+z')$

- $F = (x+y+z)(x+y'+z)(x'+y+z)(x'+y+z')$
 $= M_0 M_2 M_4 M_5$

- $F(x,y,z) = \Pi(0,2,4,5)$

Conversion between Canonical Forms

- Conversion between Canonical Forms

- $F(A,B,C) = \Sigma(1,4,5,6,7)$
- $F'(A,B,C) = \Sigma(0,2,3)$
- By DeMorgan's theorem
 $F(A,B,C) = \Pi(0,2,3)$
- $m_j' = M_j$
- sum of minterms = product of maxterms
- interchange the symbols Σ and Π and list those numbers missing from the original form
- Σ of 1's
- Π of 0's

Conversion between Canonical Forms

- Example

- $F = xy + x'z$
- $F(x, y, z) = \Sigma(1, 3, 6, 7)$
- $F(x, y, z) = \Pi(0, 2, 4, 5)$

Table 2.6

Truth Table for $F = xy + x'z$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Standard Forms

- Standard Forms

- Canonical forms are seldom used
- sum of products, SOP

$$F_1 = y' + zy + x'yz'$$

- product of sums, POS

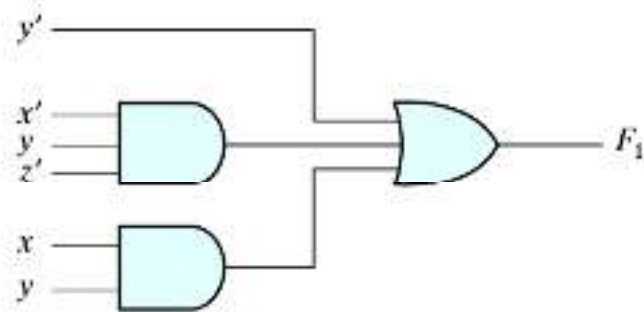
$$F_2 = x(y' + z)(x' + y + z' + w)$$

- Non standard form

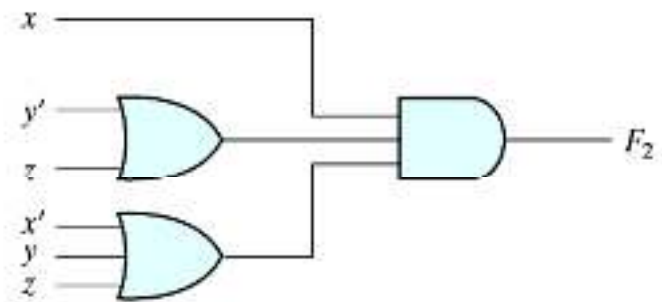
$$F_3 = AB + C(D + E)$$

Standard Forms (cont.)

- Two-level implementation

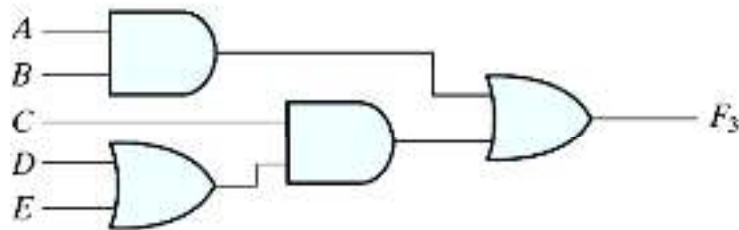


(a) Sum of Products

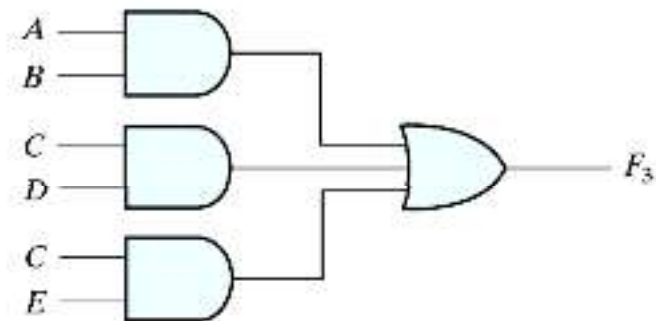


(b) Product of Sums

- Multi-level implementation



(a) $AB + C(D + E)$



(b) $AB + CD + CE$

Other Logic Operations

- 2^n rows in the truth table of n binary variables
- 2^{2n} functions for n binary variables
- 16 functions of two binary variables

Table 2.7

Truth Tables for the 16 Functions of Two Binary Variables

<i>x</i>	<i>y</i>	<i>F</i> ₀	<i>F</i> ₁	<i>F</i> ₂	<i>F</i> ₃	<i>F</i> ₄	<i>F</i> ₅	<i>F</i> ₆	<i>F</i> ₇	<i>F</i> ₈	<i>F</i> ₉	<i>F</i> ₁₀	<i>F</i> ₁₁	<i>F</i> ₁₂	<i>F</i> ₁₃	<i>F</i> ₁₄	<i>F</i> ₁₅
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

- All the new symbols except for the exclusive-OR symbol are not in common use by digital designers

Table 2.8*Boolean Expressions for the 16 Functions of Two Variables*

Boolean Functions	Operator Symbol	Name	Comments
$F_0 = 0$		Null	Binary constant 0
$F_1 = xy$	$x \cdot y$	AND	x and y
$F_2 = xy'$	x/y	Inhibition	x , but not y
$F_3 = x$		Transfer	x
$F_4 = x'y$	y/x	Inhibition	y , but not x
$F_5 = y$		Transfer	y
$F_6 = xy' + x'y$	$x \oplus y$	Exclusive-OR	x or y , but not both
$F_7 = x + y$	$x + y$	OR	x or y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$(x \oplus y)'$	Equivalence	x equals y
$F_{10} = y'$	y'	Complement	Not y
$F_{11} = x + y'$	$x \subset y$	Implication	If y , then x
$F_{12} = x'$	x'	Complement	Not x
$F_{13} = x' + y$	$x \supset y$	Implication	If x , then y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Identity	Binary constant 1

- Consider the 16 functions
 - two are equal to a constant
 - four are repeated twice
 - inhibition and implication are not commutative or associative
 - the other eight: complement, transfer, AND, OR, NAND, NOR, XOR, and equivalence are used as standard gates
 - complement: inverter
 - transfer: buffer (increasing drive strength)
 - equivalence: XNOR

Digital Logic Gates

- Boolean expression: AND, OR and NOT operations
- Constructing gates of other logic operations
 - the feasibility and economy
 - the possibility of extending gate's inputs
 - the basic properties of the binary operations
 - the ability of the gate to implement Boolean functions

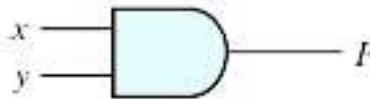
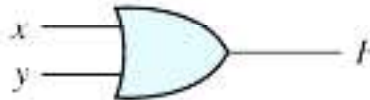
Name	Graphic symbol	Algebraic function	Truth table															
AND		$F = xy$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = x + y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	1
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$F = x'$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	x	F	0	1	1	0									
x	F																	
0	1																	
1	0																	
Buffer		$F = x$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	x	F	0	0	1	1									
x	F																	
0	0																	
1	1																	

Figure 2.5 Digital logic gates

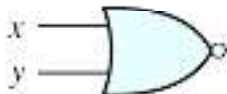
NAND		$F = (xy)'$	<table> <tr> <th>x</th> <th>y</th> <th>F</th> </tr> <tr> <td>0</td> <td>0</td> <td>1</td> </tr> <tr> <td>0</td> <td>1</td> <td>1</td> </tr> <tr> <td>1</td> <td>0</td> <td>1</td> </tr> <tr> <td>1</td> <td>1</td> <td>0</td> </tr> </table>	x	y	F	0	0	1	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = (x + y)'$	<table> <tr> <th>x</th> <th>y</th> <th>F</th> </tr> <tr> <td>0</td> <td>0</td> <td>1</td> </tr> <tr> <td>0</td> <td>1</td> <td>0</td> </tr> <tr> <td>1</td> <td>0</td> <td>0</td> </tr> <tr> <td>1</td> <td>1</td> <td>0</td> </tr> </table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	0
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	<table> <tr> <th>x</th> <th>y</th> <th>F</th> </tr> <tr> <td>0</td> <td>0</td> <td>0</td> </tr> <tr> <td>0</td> <td>1</td> <td>1</td> </tr> <tr> <td>1</td> <td>0</td> <td>1</td> </tr> <tr> <td>1</td> <td>1</td> <td>0</td> </tr> </table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Exclusive-NOR or equivalence		$F = xy + x'y'$ $= (x \oplus y)'$	<table> <tr> <th>x</th> <th>y</th> <th>F</th> </tr> <tr> <td>0</td> <td>0</td> <td>1</td> </tr> <tr> <td>0</td> <td>1</td> <td>0</td> </tr> <tr> <td>1</td> <td>0</td> <td>0</td> </tr> <tr> <td>1</td> <td>1</td> <td>1</td> </tr> </table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

Figure: Digital logic gates (continued)

Extension to multiple inputs

- Extension to multiple inputs
 - A gate can be extended to multiple inputs
 - if its binary operation is commutative and associative
 - AND and OR are commutative and associative
 - $(x+y)+z = x+(y+z) = x+y+z$
 - $(x\ y)z = x(y\ z) = x\ y\ z$

Extension to multiple inputs (cont.)

- NAND and NOR are commutative but not associative => they are not extendable

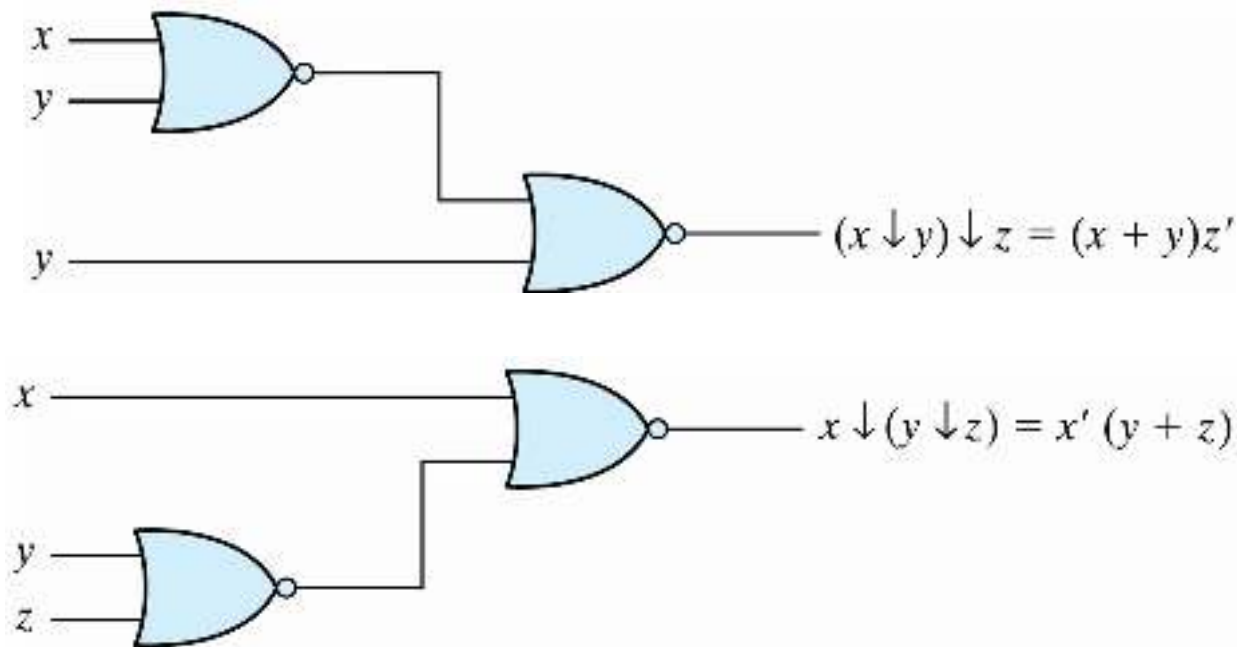
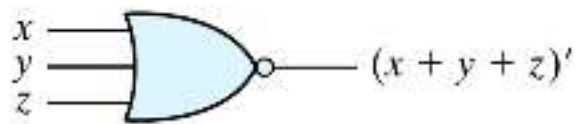


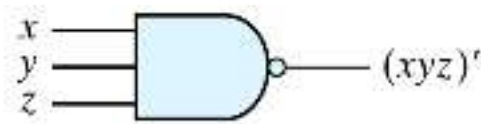
Fig. 2-6 Demonstrating the nonassociativity of the NOR operator; $(x \downarrow y) \downarrow z \neq x \downarrow (y \downarrow z)$

Extension to multiple inputs (cont.)

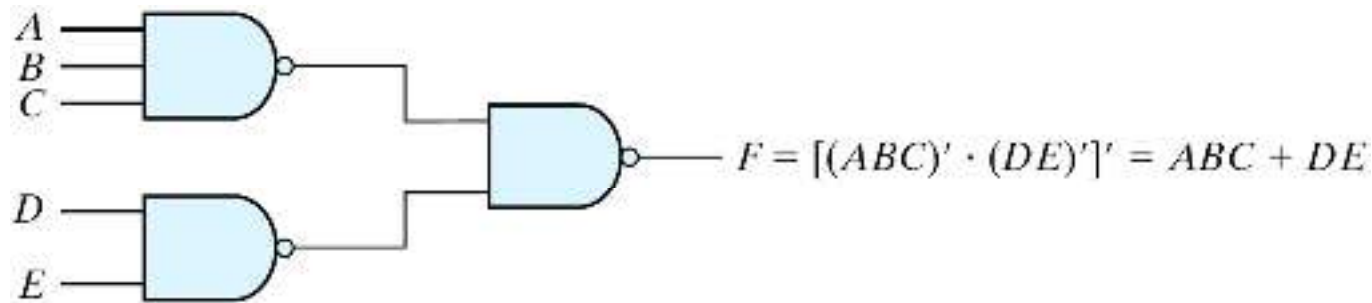
- Multiple NOR = a complement of OR gate Multiple NAND = a complement of AND
- The cascaded NAND operations = sum of products
- The cascaded NOR operations = product of sums



(a) 3-input NOR gate



(b) 3-input NAND gate

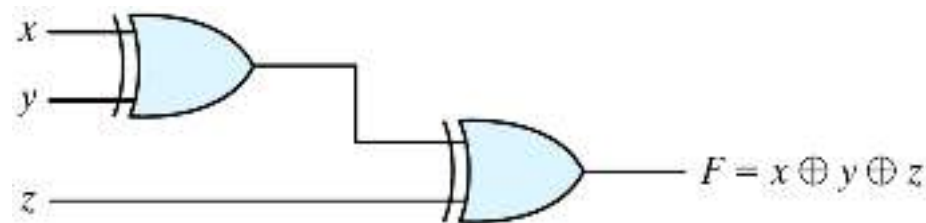


(c) Cascaded NAND gates

Fig. 2-7 Multiple-input and cascaded NOR and NAND gates

Extension to multiple inputs (cont.)

- The XOR and XNOR gates are commutative and associative
- Multiple-input XOR gates are uncommon?
- XOR is an odd function: it is equal to 1 if the inputs variables have an odd number of 1's



(a) Using 2-input gates



(b) 3-input gate

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(c) Truth table

Fig. 2-8 3-input exclusive-OR gate

Positive and Negative Logic

- Positive and Negative Logic
 - two signal values $\Leftrightarrow$ two logic values
 - positive logic: $H=1$; $L=0$
 - negative logic: $H=0$; $L=1$
- Consider a TTL gate
 - a positive logic NAND gate
 - a negative logic OR gate
 - the positive logic is used in this book

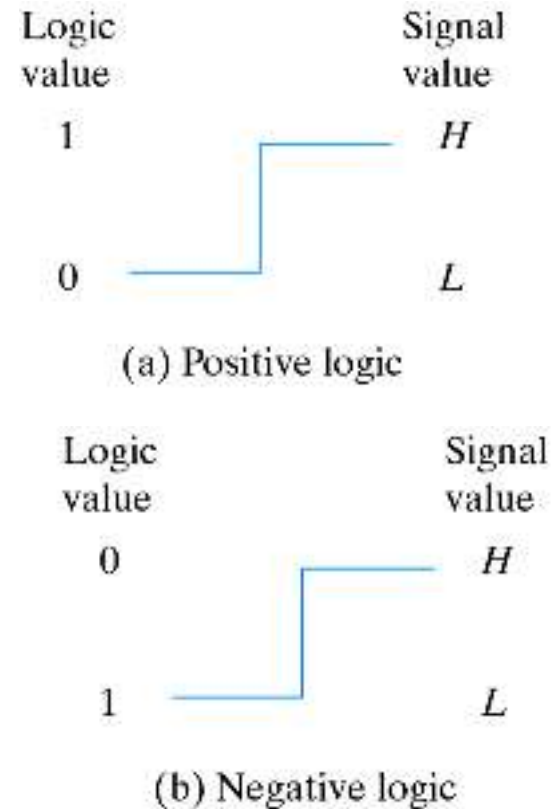


Fig. 2-9 signal assignment and logic polarity

x	y	z
L	L	L
L	H	L
H	L	L
H	H	H

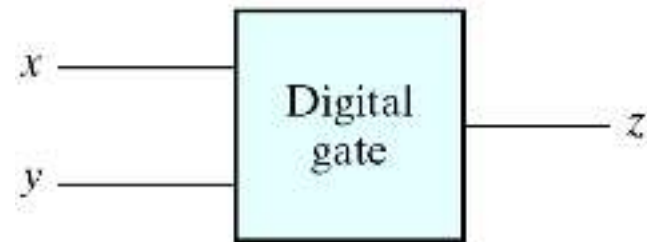
(a) Truth table with H and L

x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

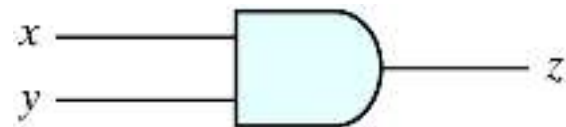
(c) Truth table for positive logic

x	y	z
1	1	1
1	0	1
0	1	1
0	0	0

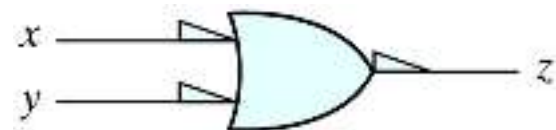
(c) Truth table for negative logic



(b) Gate block diagram



(d) Positive logic AND gate



(f) Negative logic OR gate

Fig. 2-10 Demonstration of positive and negative logic

Integrated Circuits

- An IC is a silicon semiconductor crystal, called a chip, containing the electronic components for constructing digital gates.
- Levels of integration:
 - SSI: < 10 gates
 - MSI: 10 ~ 100 gates
 - LSI: 100 ~ xk gates
 - VLSI: > xk gates
 - small size (compact size)
 - low cost
 - low power consumption
 - high reliability
 - high speed

Integrated Circuits (cont.)

- Digital logic families: circuit technology
 - TTL: transistor-transistor logic (dying?)
 - ECL: emitter-coupled logic (high speed, high power consumption)
 - MOS: metal-oxide semiconductor (NMOS, high density)
 - CMOS: complementary MOS (low power)
 - BiCMOS: high speed, high density

Integrated Circuits (cont.)

- The characteristics of digital logic families
 - *Fan-out*: the number of standard loads that the output of a typical gate can drive.
 - *Fan-in*: the number of inputs available in a gate.
 - *Power dissipation*: the power consumed by the gate.
 - *Propagation delay*: the average transition delay time for the signal to propagate from input to output.
 - *Noise margin*: the minimum of external noise voltage that caused an undesirable change in the circuit output.

Integrated Circuits (cont.)

- CAD – Computer-Aided Design
 - Millions of transistors
 - Computer-based representation and aid
 - Automatic the design process
 - Design entry
 - Schematic capture
 - HDL – Hardware Description Language
 - Verilog, VHDL
 - Simulation
 - Physical realization
 - ASIC, FPGA, PLD

Reading:

Page No. 51-68, Chapter 2

Digital Design with an Introduction to Verilog HDL, 5th Edition by Morris Mano

Thanks!